arXiv:2406.11944v2 [cs.LG] 6 Nov 2024

Transcoders Find Interpretable LLM Feature Circuits

Jacob Dunefsky[*]
Yale University
New Haven, CT 06511
jacob.dunefsky@yale.edu

Philippe Chlenski[*]
Columbia University
New York, NY 10027
pac@cs.columbia.edu

Neel Nanda

Abstract

A key goal in mechanistic interpretability is circuit analysis: finding sparse subgraphs of models corresponding to specific behaviors or capabilities. However, MLP sublayers make fine-grained circuit analysis on transformer-based language models difficult. In particular, interpretable features—such as those found by sparse autoencoders (SAEs)—are typically linear combinations of extremely many neurons, each with its own nonlinearity to account for. Circuit analysis in this setting thus either yields intractably large circuits or fails to disentangle local and global behavior. To address this we explore **transcoders**, which seek to faithfully approximate a densely activating MLP layer with a wider, sparsely-activating MLP layer. We introduce a novel method for using transcoders to perform weights-based circuit analysis through MLP sublayers. The resulting circuits neatly factorize into input-dependent and input-invariant terms. We then successfully train transcoders on language models with 120M, 410M, and 1.4B parameters, and find them to perform at least on par with SAEs in terms of sparsity, faithfulness, and human-interpretability. Finally, we apply transcoders to reverse-engineer unknown circuits in the model, and we obtain novel insights regarding the "greater-than circuit" in GPT2-small. Our results suggest that transcoders can prove effective in decomposing model computations involving MLPs into interpretable circuits. Code is available at https://github.com/jacobdunefsky/transcoder_circuits/.

1 Introduction

In recent years, transformer-based large language models (LLMs) have displayed outstanding performance on a wide variety of tasks [8, 43, 46]. However, the mechanisms by which LLMs perform these tasks are opaque by default [10, 33]. The field of mechanistic interpretablity [9] seeks to understand these mechanisms, and doing so relies on decomposing a model into **circuits** [41]: interpretable subcomputations responsible for specific model behaviors [15, 32, 42, 50].

A core problem in fine-grained circuit analysis is incorporating MLP sublayers [32, 38]. Attempting to analyze MLP neurons directly suffers from "polysemanticity" [3, 16, 24, 40]: the tendency of neurons to activate on many unrelated concepts. To address this, **sparse autoencoders (SAEs)** [7, 12, 51] have been used to perform fine-grained circuit analysis by instead looking at **features**—vectors in the model's representation space—instead of individual neurons [14, 34]. However, while SAE features are often interpretable, these vectors tend to be dense linear combinations of many neurons [36]. Thus, mechanistically understanding how an SAE feature before one or more MLP layers affects a later SAE feature may require considering an infeasible number of neurons and their nonlinearities. Prior attempts to circumvent this [14, 34] use a mix of causal interventions and gradient-based

[*]Equal contribution.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

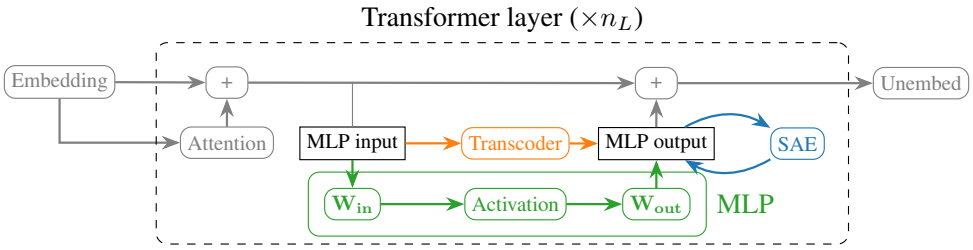

Figure 1: A comparison between **SAEs**, **MLP transcoders**, and **MLP sublayers** for a transformer-based language model. SAEs learn to reconstruct model activations, whereas transcoders imitate sublayers' input-output behavior.

approximations to MLP layers. But these approaches fail to exhibit **input-invariance**: the connections between features can only ever be described *for a given input*, and not for the model as a whole. Attempts to address this, e.g. by averaging results over many inputs, conversely lose their ability to yield **input-dependent** information that describes a connection's importance on a single input. This means that SAEs *cannot tell us about the general input-output behavior of an MLP across all inputs*.

To address why input-invariance is desirable, consider the following example: say that one has a post-MLP SAE feature and wants to see how it is computed from pre-MLP SAE features. Doing e.g. patching on one input shows that a pre-MLP feature for Polish last names is important for causing the post-MLP feature to activate. But on other inputs, would features other than the Polish last name feature also cause the post-MLP feature to fire (e.g. an English last names feature)? Could there be other inputs where the Polish last names feature fires but the post-MLP feature does not? We can see that without input-invariance, it is difficult to make general claims about model behavior.

Motivated by this, in this work, we explore **transcoders** (an idea proposed, but not explored, in Templeton et al. [47] and Li et al. [31]): wide, sparsely-activating approximations of a model's original MLP sublayer. Specifically, MLP transcoders are wide ReLU MLPs with one hidden layer that are trained to faithfully approximate the original narrower MLP sublayer's output, with an L1 regularization penalty on neuron activations to encourage sparse activations. **Our primary motivation** is to enable input-invariant feature-level circuit analysis through MLP sublayers, which allows us to understand and interpret the general behavior of circuits involving MLP sublayers.

Our contributions. Our main contributions are (1) to introduce a method for circuit analysis using transcoders, (2) to confirm that transcoders are a faithful and interpretable approximation to MLP sublayers, and (3) to demonstrate the utility of our circuit analysis method on detailed case studies.

After describing the architecture of transcoders in §3.1, we demonstrate in §3.2 that transcoders additionally enable circuit-finding techniques that are not possible using SAEs, and introduce a novel method for performing circuit analysis with transcoders and demonstrate that transcoders cleanly factorize circuits into *input-invariant* and *input-dependent* components.

Then, in §4, we evaluate transcoders' interpretability, sparsity, and faithfulness to the original model. Because SAEs are the standard method for finding sparse decompositions of model activations, we compare transcoders to SAEs on models up to 1.4 billion parameters and verify that transcoders are on par with SAEs or better with respect to these properties.

We apply transcoder circuit analysis to a variety of tasks in §5.1 and §5.2, including "blind case studies," which demonstrate how this approach allows us to understand features without looking at specific examples, and an in-depth analysis of the GPT2-small "greater-than circuit" previously studied by Hanna et al. [26].

2 Transformers preliminaries

Following Elhage et al. [15], we represent the computation of a transformer model as follows. First, the model maps input tokens (and their positions) to embeddings $\mathbf{x_{pre}^{(0,t)}} \in \mathbb{R}^{d_{\text{model}}}$, where t is the token index and d_{model} is the *model dimensionality*. Then, the model applies a series of "layers," which map the hidden state at the end of the previous block to the new hidden state. This can be expressed as:

$$\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})} = \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})} + \sum_{\text{head } h} \text{attn}^{(l,h)} \left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}; \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{1}:\mathbf{t})} \right) \tag{1}$$

$$\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l}+\mathbf{1},\mathbf{t})} = \mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})} + \text{MLP}^{(l)} \left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})} \right) \tag{2}$$

where l is the layer index, t is the token index, $\text{attn}^{(l,h)}(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}; \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{1}:\mathbf{t})})$ denotes the output of attention head h at layer l given all preceding source tokens $\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{1}:\mathbf{t})}$ and destination token $\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}$, and $\text{MLP}^{(l)}(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})})$ denotes the output of the layer l MLP.[2]

Equation 1 shows how the **attention sublayer** updates the hidden state at token t, and Equation 2 shows how the **MLP sublayer** updates the hidden state. Importantly, each sublayer always *adds* its output to the current hidden state. As such, the hidden state always can be additively decomposed into the outputs of all previous sublayers. This motivates Elhage et al. [15] to refer to each token's hidden state as its **residual stream**, which is "read from" and "written to" by each sublayer.

3 Transcoders

3.1 Architecture and training

Transcoders aim to learn a "sparsified" approximation of an MLP sublayer: they approximate the output of an MLP sublayer as a sparse linear combination of feature vectors. Formally, the transcoder architecture can be expressed as

$$\mathbf{z}_{\mathbf{TC}}(\mathbf{x}) = \text{ReLU}\left(\mathbf{W}_{\mathbf{enc}}\mathbf{x} + \mathbf{b}_{\mathbf{enc}}\right) \tag{3}$$

$$\text{TC}(\mathbf{x}) = \mathbf{W}_{\mathbf{dec}}\mathbf{z}_{\mathbf{TC}}(\mathbf{x}) + \mathbf{b}_{\mathbf{dec}}, \tag{4}$$

where $\mathbf{x}$ is the input to the MLP sublayer, $\mathbf{W}_{\mathbf{enc}} \in \mathbb{R}^{d_{\text{features}} \times d_{\text{model}}}$, $\mathbf{W}_{\mathbf{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{features}}}$, $\mathbf{b}_{\mathbf{enc}} \in \mathbb{R}^{d_{\text{features}}}$, $\mathbf{b}_{\mathbf{dec}} \in \mathbb{R}^{d_{\text{model}}}$, d_{features} is the number of feature vectors in the transcoder, and d_{model} is the dimensionality of the MLP input activations. Usually, d_{features} is far greater than d_{model}.

Each feature in a transcoder is associated with two vectors: the i-th row of $\mathbf{W}_{\mathbf{enc}}$ is the **encoder feature vector** of feature i, and the i-th column of $\mathbf{W}_{\mathbf{dec}}$ is the **decoder feature vector** of feature i. The i-th component of $\mathbf{z}_{\mathbf{TC}}(\mathbf{x})$ is called the **activation** of feature i. Intuitively, for each feature, the encoder vector is used to determine how much the feature should activate; the decoder vector is then scaled by this amount, and the resulting weighted sum of decoder vectors is the output of the transcoder. In this paper, the notation $\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l},\mathbf{i})}$ and $\mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{i})}$ is used to denote the i-th encoder feature vector and decoder feature vector, respectively, in the layer l transcoder.

Because we want transcoders to learn to approximate an MLP sublayer's computation with a sparse linear combination of feature vectors, transcoders are trained with the following loss, where λ_1 is a hyperparameter mediating the tradeoff between sparsity and faithfulness:

$$\mathcal{L}_{TC}(\mathbf{x}) = \underbrace{\|\text{MLP}(\mathbf{x}) - \text{TC}(\mathbf{x})\|_2^2}_{\text{faithfulness loss}} + \underbrace{\lambda_1 \|\mathbf{z}_{\mathbf{TC}}(\mathbf{x})\|_1}_{\text{sparsity penalty}}. \tag{5}$$

3.2 Circuit analysis with transcoders

We now introduce a novel method for performing feature-level circuit analysis with transcoders, which provides a scalable and interpretable way to identify which transcoder features in different layers connect to compute a given task. Importantly, this method provides insights into the general input-output behavior of MLP sublayers, which SAE-based methods cannot do.

In particular, the primary goal of circuit analysis is to identify a subgraph of the model's computational graph that is responsible for (most of) the model's behavior on a given task [11, 19, 20]; this requires a means of evaluating a computational subgraph's importance to the task in question. In order to

[2] Note that the "Pythia" family of models computes MLP and attention sublayer outputs in parallel. This means that Equation 2 is thus given by $\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l}+\mathbf{1},\mathbf{t})} = \mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})} + \text{MLP}^{(l)} \left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})} \right)$.